**Distributed Pizza Ordering System on AWS**

---

## 📌 Project Overview

The **Distributed Pizza Ordering System** is a microservices-based asynchronous architecture built using **AWS services — API Gateway, Lambda, SQS, and EC2**. It simulates a real-world food ordering system where users place orders through a web interface, which are then processed asynchronously by EC2 worker instances.

This project demonstrates **cloud-native design**, **event-driven architecture**, and **asynchronous message processing** — key principles in **DevOps and distributed systems**.

---

## 🎯 Project Goals

- To design a **scalable and decoupled** pizza ordering system using AWS services.
- To enable **real-time order submission** via a web interface and **asynchronous processing** by EC2 workers.
- To showcase integration between **API Gateway, Lambda, SQS, and EC2**.
- To simulate a **real-world distributed microservice environment**.

---

## 🧩 System Architecture

### Architecture Components

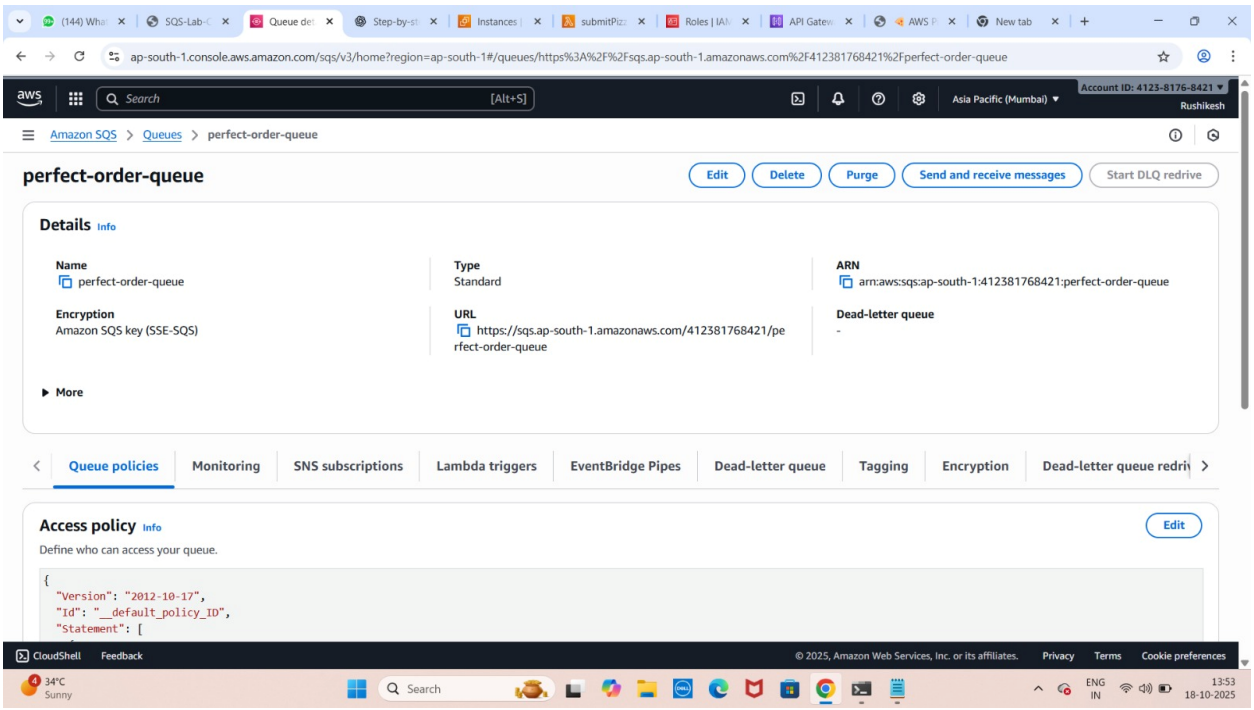
1. **Frontend (EC2 Web Server)** – Hosts the HTML order form where users submit pizza orders.
2. **API Gateway** – Handles incoming HTTP requests and routes them to Lambda.
3. **AWS Lambda** – Processes incoming requests and sends messages to SQS.
4. **SQS Queue** – Acts as a buffer holding pending pizza orders.
5. **EC2 Worker Instance(s)** – Continuously polls SQS and processes each order asynchronously.

---

## ⚙️ Step-by-Step Implementation

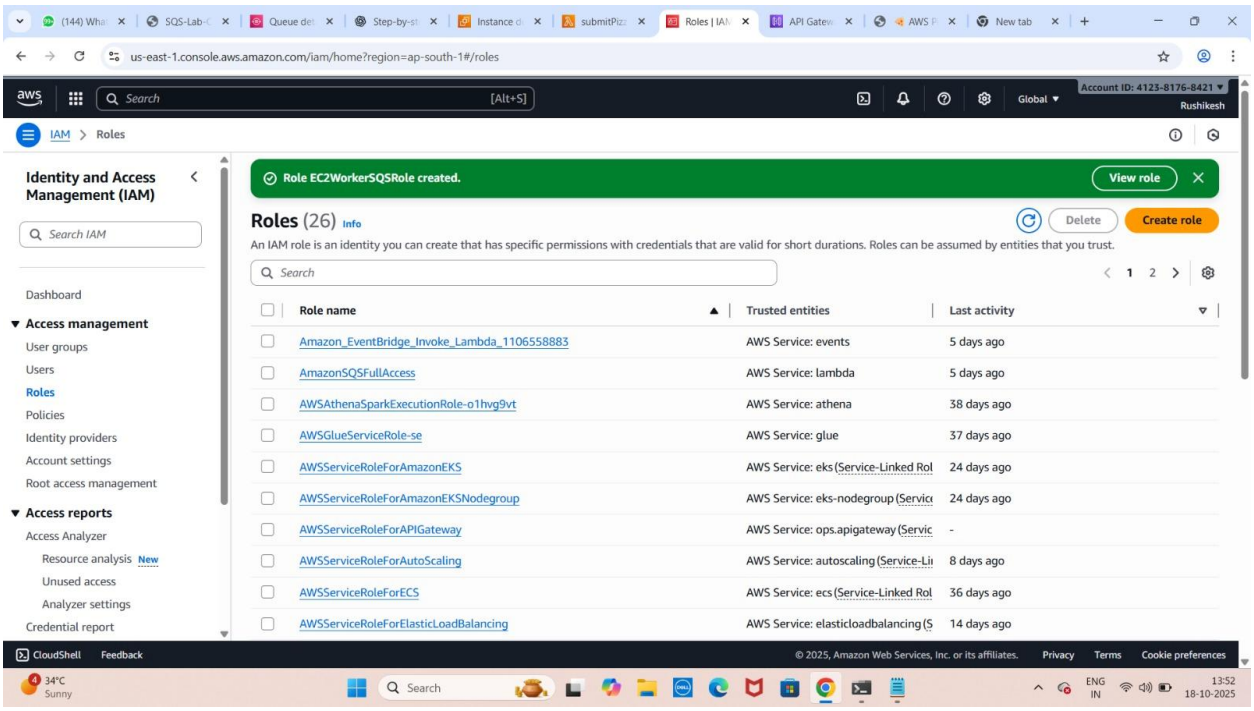
**Step 1: Create SQS Queue**

- Navigate to **Amazon SQS → Create Queue**.
- Name: perfect-order-queue
- Note the SQS URL:
- https://sqs.<region>.amazonaws.com/<account_id>/perfect-order-queue




**Step 2: Create Lambda Role**

- Go to **IAM → Roles → Create Role**.
- Choose **Lambda** as the trusted entity.
- Attach policy: **AmazonSQSFullAccess**.
- Name the role LambdaSQSRole.




---

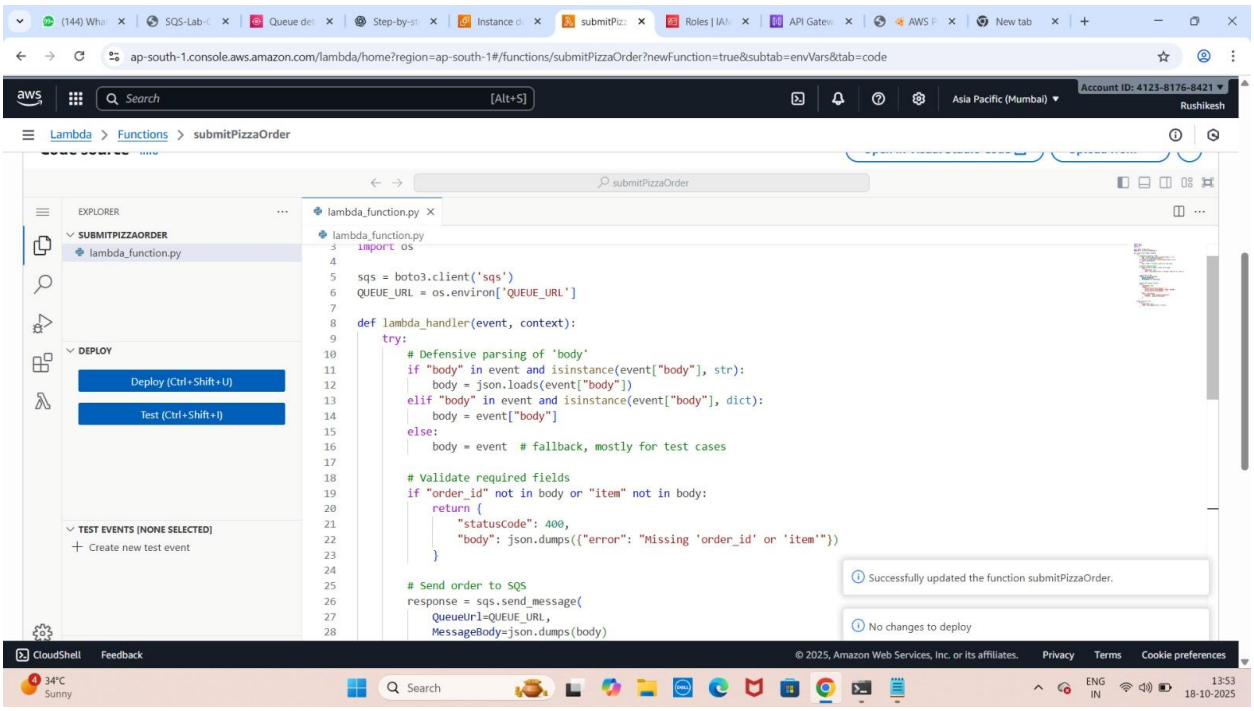
**Step 3: Create Lambda Function**

- Create a new **Lambda function** using Python 3.x.
- Assign the role LambdaSQSRole.
- Add the environment variable:
    - **Key**: QUEUE_URL
    - **Value**: SQS Queue URL

📜 **Lambda Code:**

```
import json, boto3, os


sqs = boto3.client('sqs')

QUEUE_URL = os.environ['QUEUE_URL']


def lambda_handler(event, context):

    body = json.loads(event["body"])
```

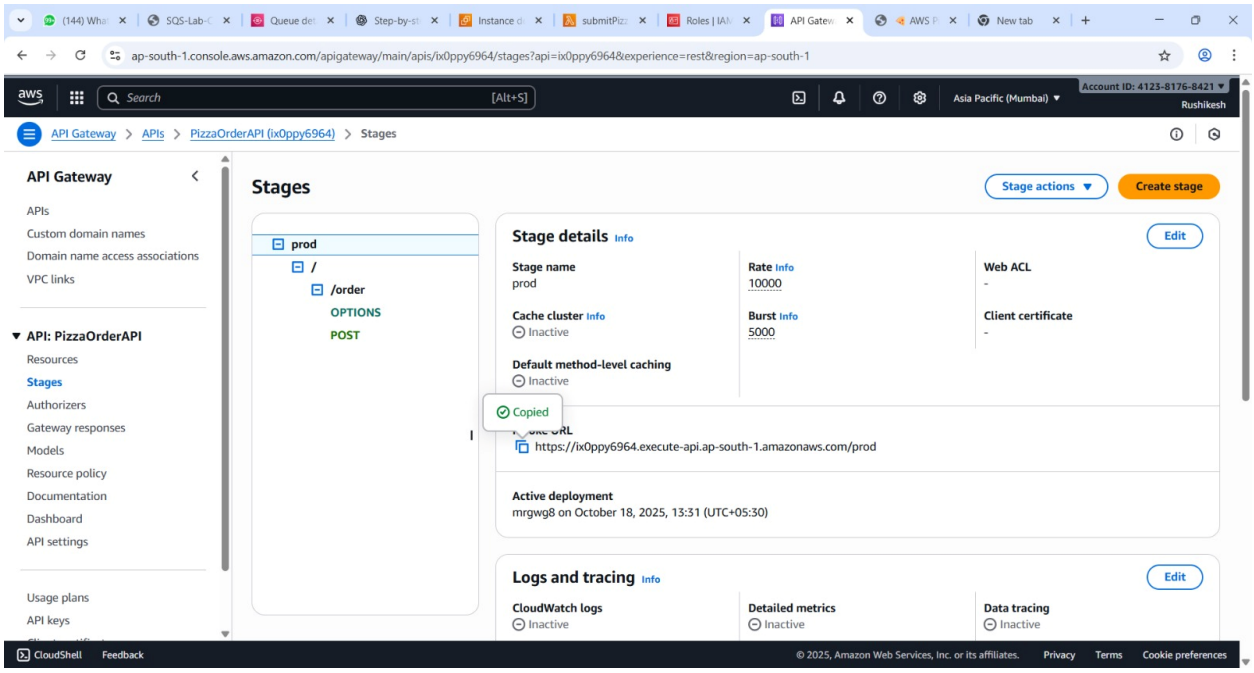
```
response = sqs.send_message(
    QueueUrl=QUEUE_URL,
    MessageBody=json.dumps(body)
)
return {
    "statusCode": 200,
    "headers": {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Methods": "POST, OPTIONS",
        "Access-Control-Allow-Headers": "*"
    },
    "body": json.dumps({
        "message": "Order placed successfully!",
        "orderId": response["MessageId"]
    })
}
```




---

**Step 4: Create API Gateway**

- Create a **REST API** in **Amazon API Gateway**.
- Resource: /order
- Method: POST → Integrate with Lambda.
- Enable **CORS**.
- Deploy API to a stage named prod.
- Note the Invoke URL:
- https://<api-id>.execute-api.<region>.amazonaws.com/prod/order




---

**Step 5: EC2 Worker Setup**

- Launch an **Amazon Linux 2 EC2 instance**.
- Attach an **IAM Role** with **AmazonSQSFullAccess**.
- Run the following setup script:

```
#!/bin/bash

yum update -y

yum install -y python3

python3 -m pip install --upgrade pip boto3


cat > /home/ec2-user/worker.py << 'EOF'

import boto3, json, time, socket, random


sqs = boto3.client("sqs", region_name="us-east-1")

QUEUE_URL = "https://sqs.us-east-1.amazonaws.com/<account_id>/perfect-order-queue"
```

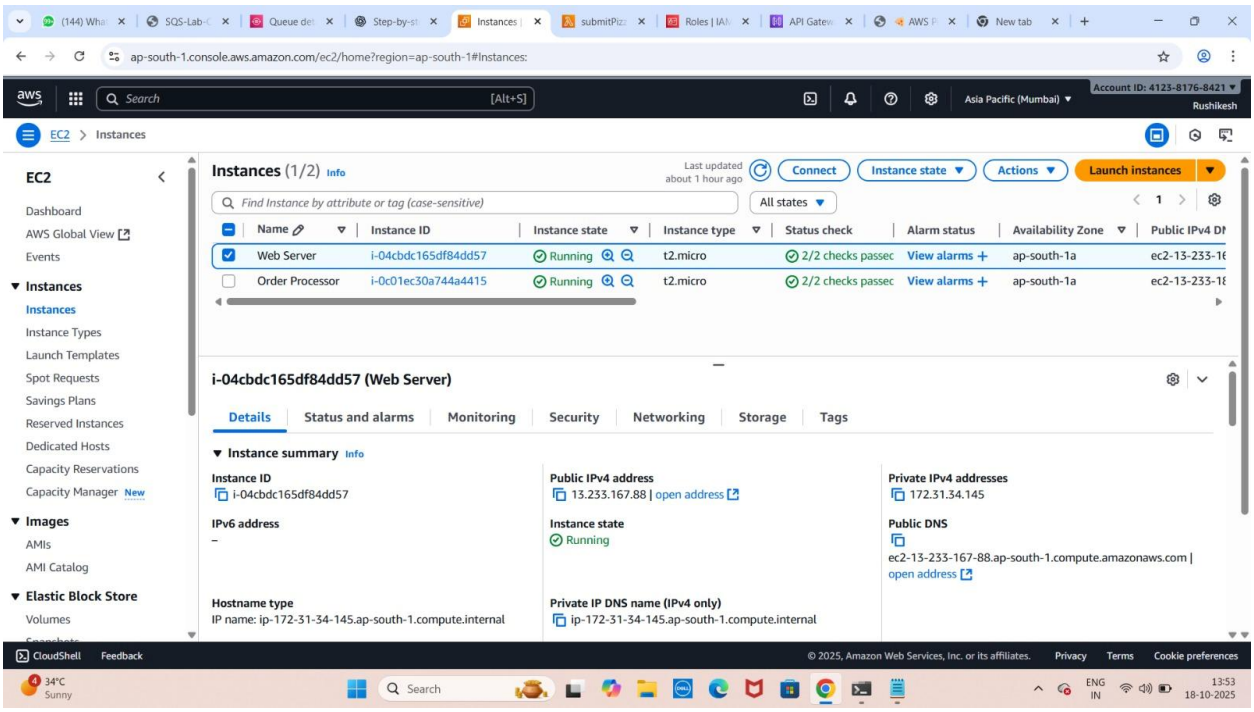
```
hostname = socket.gethostname()


def process(order):
    print(f" 🍕 {hostname} is cooking: {order}", flush=True)
    time.sleep(random.randint(1, 3))


while True:
    try:
        response = sqs.receive_message(
            QueueUrl=QUEUE_URL,
            MaxNumberOfMessages=1,
            WaitTimeSeconds=10
        )
        messages = response.get("Messages", [])
        if messages:
            for msg in messages:
                order = json.loads(msg["Body"])
                process(order)
                sqs.delete_message(
                    QueueUrl=QUEUE_URL,
                    ReceiptHandle=msg["ReceiptHandle"]
                )
        else:
            print(f" 😴 {hostname} — No new orders. Waiting...", flush=True)
    except Exception as e:
        print(f" ❌ {hostname} — Error: {e}", flush=True)
```

time.sleep(1)

EOF

cd /home/ec2-user

nohup python3 worker.py >> worker.log 2>&1 &




---

**Step 6: Frontend (Pizza Order Page)**

Host the frontend on another EC2 instance running Apache (httpd).

#!/bin/bash

yum update -y

yum install -y httpd

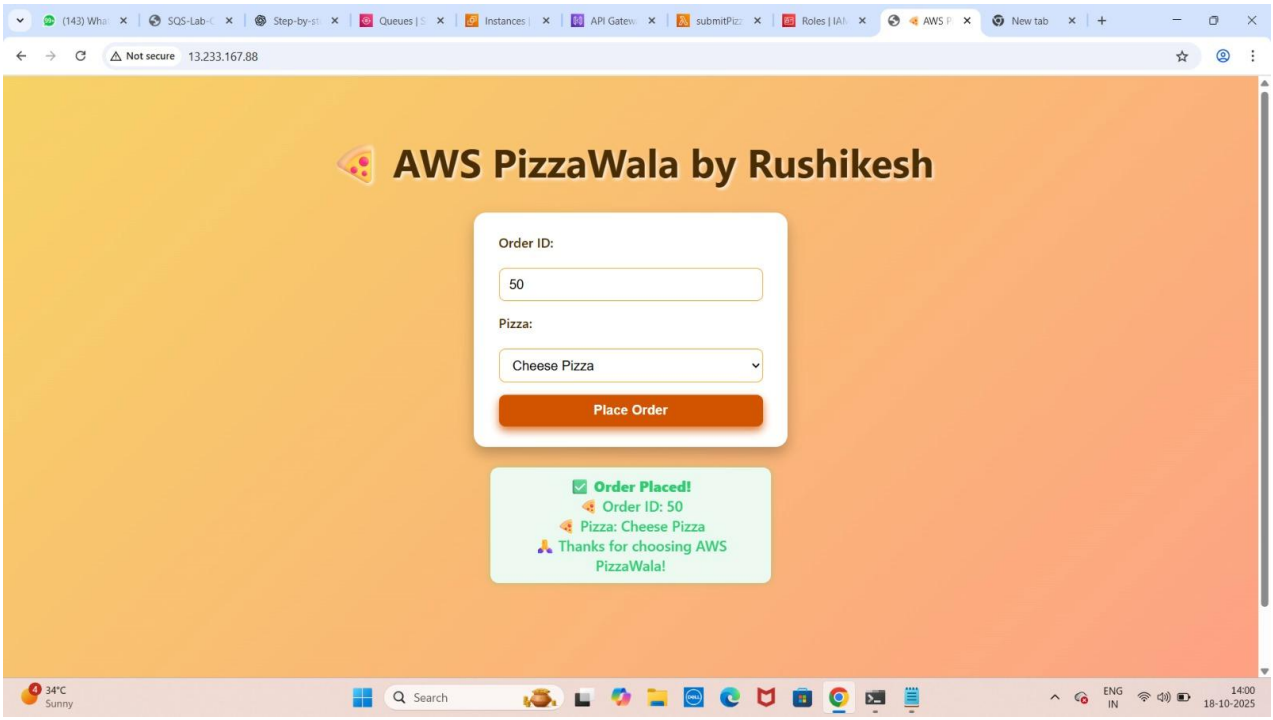
systemctl start httpd

systemctl enable httpd

📜 **index.html (Frontend UI)**

- Simple HTML form to take pizza orders.

- Sends POST request to the API Gateway endpoint.
- Displays confirmation message after successful order.

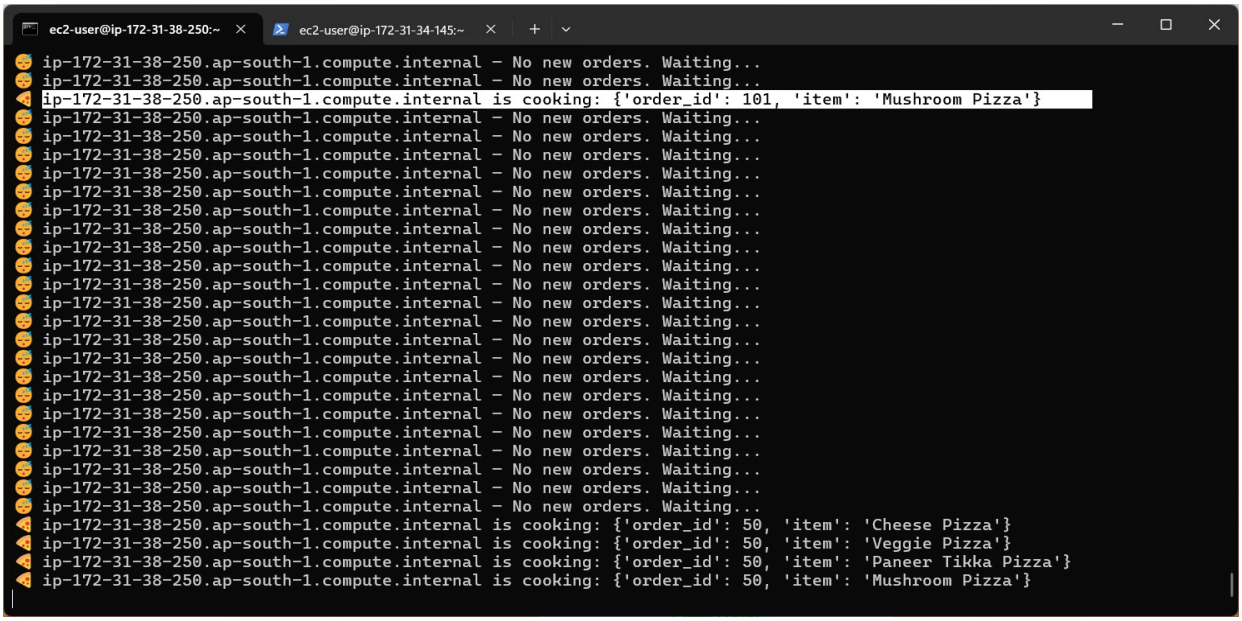



---

## Useful Commands

```
# Start worker

nohup python3 worker.py >> worker.log 2>&1 &


# View logs

tail -f worker.log


# Stop worker

pkill -f worker.py


# Send multiple test orders

for i in {1..10}; do
```

```
curl -X POST "https://<api-id>.execute-api.<region>.amazonaws.com/prod/order" \
-H "Content-Type: application/json" \
-d "{\"order_id\": $i, \"item\": \"Cheese Pizza\"}" &
done
```




---

## 🧠 Key Concepts Demonstrated

### AWS SQS

- Acts as a buffer between producer (Lambda) and consumer (EC2).
- Enables decoupling and horizontal scalability.
- Ensures reliable message delivery.

### AWS Lambda

- Stateless and event-driven.
- Bridges API Gateway and SQS for asynchronous processing.
- Scalable, pay-per-execution model.

### Amazon API Gateway

- Handles REST API requests and routes them to Lambda.

- Provides throttling, logging, and CORS support.

**Amazon EC2 Workers**

- Continuously poll the SQS queue.
- Simulate real-time pizza order processing.
- Easily scalable using Auto Scaling Groups.

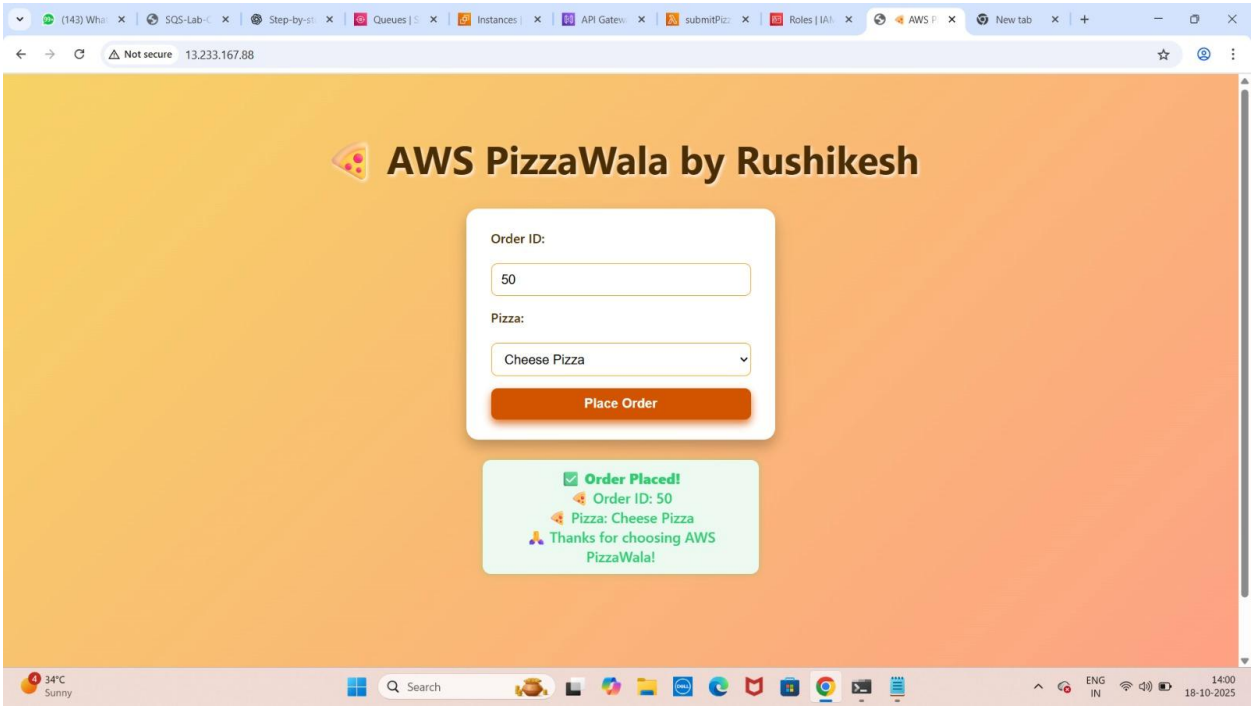
**Asynchronous Architecture**

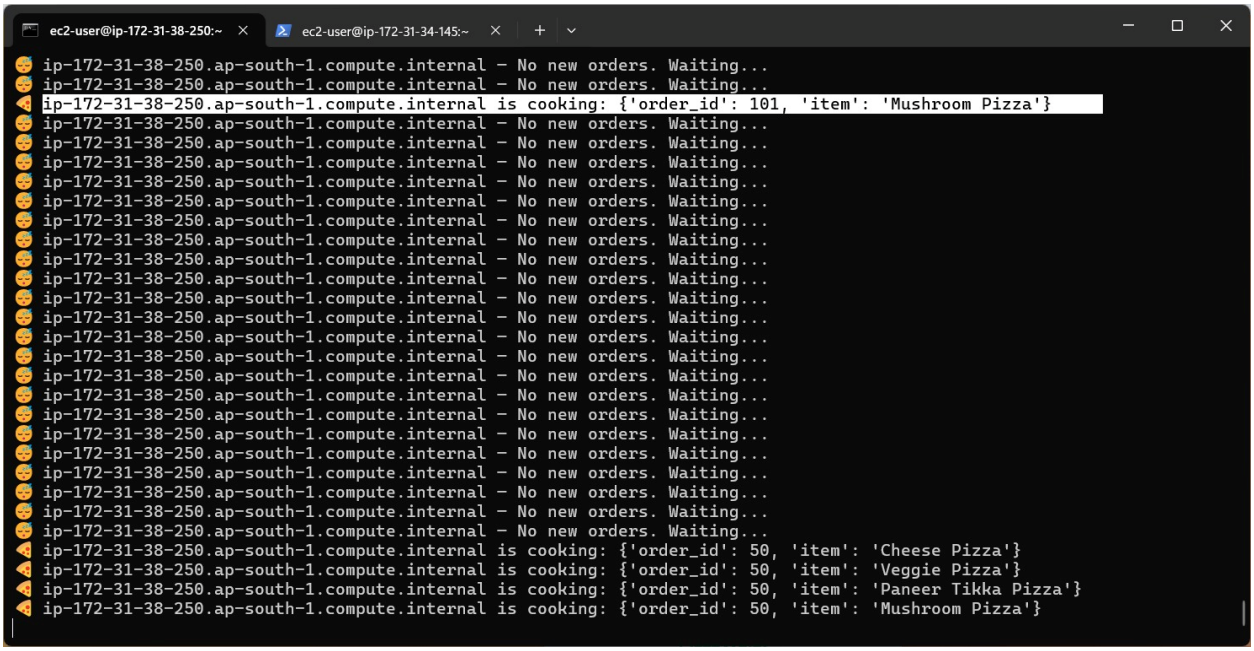
- Orders are queued and processed later.
- Improves responsiveness and reliability.
- Ideal for burst workloads or distributed environments.

---

## 📈 Project Outcomes

✅ Demonstrated end-to-end integration of AWS managed services.
✅ Built a fully **asynchronous event-driven system**.
✅ Achieved **loose coupling and high scalability**.
✅ Gained hands-on experience in **API Gateway, Lambda, SQS, EC2, and IAM**.






---

## 🧰 Technologies Used

| Category | Tools/Services |
|---|---|
| **Cloud Platform** | AWS |
| **Compute** | EC2 |
| **Messaging** | Amazon SQS |
| **Serverless** | AWS Lambda |
| **API Management** | API Gateway |
| **Frontend** | HTML, JavaScript |
| **Programming Language** | Python |
| **IAM Permissions** | AmazonSQSFullAccess |

---

## 📚 Learning & Takeaways

- Learned about **event-driven architectures** and **asynchronous workflows**.
- Understood **decoupling** between frontend, backend, and workers.

- Explored **AWS managed service integrations**.
- Enhanced skills in **Python automation, AWS Lambda, and SQS message handling**.

---